

19.1. Pipes

Pipes are an interprocess communication mechanism that is provided in all flavors of Unix. A pipe is a one-way flow of data between processes: all data written by a process to the pipe is routed by the kernel to another process, which can thus read it.

In Unix command shells, pipes can be created by means of the `|` operator. For instance, the following statement instructs the shell to create two processes connected by a pipe:

```
$ ls | more
```

The standard output of the first process, which executes the *ls* program, is redirected to the pipe; the second process, which executes the *more* program, reads its input from the pipe.

Note that the same results can also be obtained by issuing two commands such as the following:

```
$ ls > temp
$ more < temp
```

The first command redirects the output of *ls* into a regular file; then the second command forces *more* to read its input from the same file. Of course, using pipes instead of temporary files is usually more convenient due to the following reasons:

- The shell statement is much shorter and simpler.
- There is no need to create temporary regular files, which must be deleted later.

19.1.1. Using a Pipe

Pipes may be considered open files that have no corresponding image in the mounted filesystems. A process creates a new pipe by means of the `pipe( )` system call, which returns a pair of file descriptors; the process may then pass these descriptors to its descendants through `fork( )`, thus sharing the pipe with them. The processes can read from the pipe by using the `read( )` system call with the first file descriptor; likewise, they can write into the pipe by using the `write( )` system call with the second file descriptor.

POSIX defines only half-duplex pipes, so even though the `pipe( )` system call returns two file descriptors, each process must close

one before using the other. If a two-way flow of data is required, the processes must use two different pipes by invoking `pipe ( )` twice.

Several Unix systems, such as System V Release 4, implement full-duplex pipes . In a full-duplex pipe, both descriptors can be written into and read from, thus there are two bidirectional channels of information. Linux adopts yet another approach: each pipe's file descriptors are still one-way, but it is not necessary to close one of them before using the other.

Let's resume the previous example. When the command shell interprets the `ls|more` statement, it essentially performs the following actions:

1. Invokes the `pipe ( )` system call; let's assume that `pipe ( )` returns the file descriptors 3 (the pipe's read channel) and 4 (the write channel).
2. Invokes the `fork ( )` system call twice.
3. Invokes the `close ( )` system call twice to release file descriptors 3 and 4.

The first child process, which must execute the `ls` program, performs the following operations:

1. Invokes `dup2 (4, 1)` to copy file descriptor 4 to file descriptor 1. From now on, file descriptor 1 refers to the pipe's write channel.
2. Invokes the `close ( )` system call twice to release file descriptors 3 and 4.
3. Invokes the `execve ( )` system call to execute the `ls` program (see the section "[The exec Functions](#)" in [Chapter 20](#)). The program writes its output to the file that has file descriptor 1 (the standard output); i.e., it writes into the pipe.

The second child process must execute the `more` program; therefore, it performs the following operations:

1. Invokes `dup2 (3, 0)` to copy file descriptor 3 to file descriptor 0. From now on, file descriptor 0 refers to the pipe's read channel.
2. Invokes the `close ( )` system call twice to release file descriptors 3 and 4.
3. Invokes the `execve ( )` system call to execute `more`. By default, that program reads its input from the file that has file descriptor 0 (the standard input); i.e., it reads from the pipe.

In this simple example, the pipe is used by exactly two processes. Because of its implementation, though, a pipe can be used by an arbitrary number of processes.^[*] Clearly, if two or more processes read or write the same pipe, they must explicitly synchronize their accesses by using file locking (see the section "[Linux File](#)

Locking" in Chapter 12) or IPC semaphores (see the section "IPC Semaphores" later in this chapter).

[*] Because most shells offer pipes that connect only two processes, applications requiring pipes used by more than two processes must be coded in a programming language such as C.

Many Unix systems provide, besides the `pipe( )` system call, two wrapper functions named `popen( )` and `pclose( )` that handle all the dirty work usually done when using pipes. Once a pipe has been created by means of the `popen( )` function, it can be used with the high-level I/O functions included in the C library (`fprintf( )`, `fscanf( )`, and so on).

In Linux, `popen( )` and `pclose( )` are included in the C library. The `popen( )` function receives two parameters: the `filename` pathname of an executable file and a `type` string specifying the direction of the data transfer. It returns the pointer to a `FILE` data structure. The `popen( )` function essentially performs the following operations:

1. Creates a new pipe by using the `pipe( )` system call.
2. Forks a new process, which in turn executes the following operations:
 - a. If `type` is `r`, it duplicates the file descriptor associated with the pipe's write channel as file descriptor 1 (standard output); otherwise, if `type` is `w`, it duplicates the file descriptor associated with the pipe's read channel as file descriptor 0 (standard input).
 - b. Closes the file descriptors returned by `pipe( )`.
 - c. Invokes the `execve( )` system call to execute the program specified by `filename`.
3. If `type` is `r`, it closes the file descriptor associated with the pipe's write channel; otherwise, if `type` is `w`, it closes the file descriptor associated with the pipe's read channel.
4. Returns the address of the `FILE` file pointer that refers to whichever file descriptor for the pipe is still open.

After the `popen( )` invocation, parent and child can exchange information through the pipe: the parent can read (if `type` is `r`) or write (if `type` is `w`) data by using the `FILE` pointer returned by the function. The data is written to the standard output or read from the standard input, respectively, by the program executed by the child process.

The `pclose( )` function (which receives the file pointer returned by `popen( )` as its parameter) simply invokes the `wait4( )` system call and waits for the termination of the process created by `popen( )`.

19.1.2. Pipe Data Structures

We now have to start thinking again at the system call level. Once a pipe is created, a process uses the `read( )` and `write( )` VFS system calls to access it. Therefore, for each pipe, the kernel creates an inode object plus two file objects one for reading and the other for writing. When a process wants to read from or write to the pipe, it must use the proper file descriptor.

When the inode object refers to a pipe, its `i_pipe` field points to a `pipe_inode_info` structure shown in [Table 19-1](#).

Table 19-1. The `pipe_inode_info` structure

Type	Field	Description
<code>struct wait_queue *</code>	<code>wait</code>	Pipe/FIFO wait queue
<code>unsigned int</code>	<code>nrbufs</code>	Number of buffers containing data to be read
<code>unsigned int</code>	<code>curbuf</code>	Index of first buffer containing data to be read
<code>struct pipe_buffer [16]</code>	<code>bufs</code>	Array of pipe's buffer descriptors
<code>struct page *</code>	<code>tmp_page</code>	Pointer to a cached page frame
<code>unsigned int</code>	<code>start</code>	Read position in current pipe buffer
<code>unsigned int</code>	<code>readers</code>	Flag for (or number of) reading processes
<code>unsigned int</code>	<code>writers</code>	Flag for (or number of) writing processes
<code>unsigned int</code>	<code>waiting_writers</code>	Number of writing processes sleeping in the wait queue
<code>unsigned int</code>	<code>r_counter</code>	Like <code>readers</code> , but used when waiting for a process that reads from the FIFO
<code>unsigned int</code>	<code>w_counter</code>	Like <code>writers</code> , but used when waiting for a process that writes into the FIFO
<code>struct fasync_struct *</code>	<code>fasync_readers</code>	Used for asynchronous I/O notification via signals
<code>struct</code>	<code>fasync_writers</code>	

fasync_struct *

Used for
asynchronous I/O
notification via
signals

Besides one inode and two file objects, each pipe has its own set of pipe buffers . Essentially, a pipe buffer is a page frame that contains data written into the pipe and yet to be read. Up to Linux 2.6.10, each pipe had just one pipe buffer. In the 2.6.11 kernel, however, data buffering for pipes (and FIFOs) has been heavily revised, and now each pipe makes use of 16 pipe buffers. This change greatly enhances the performance of User Mode applications that write large chunks of data in a pipe.

The `bufs` field of the `pipe_inode_info` data structure stores an array of 16 `pipe_buffer` objects, each of which describes a pipe buffer. The fields of this object are shown in [Table 19-2](#).

Table 19-2. The fields of the pipe_buffer object

Type

Field

Description

`struct page *`

`page`

Address of the descriptor of the page frame for the pipe buffer

`unsigned int`

`offset`

Current position of the significant data inside the page frame

`unsigned int`

`len`

Length of the significant data in the pipe buffer

`struct`

`pipe_buf_operations *`

`ops`

Address of a table of methods relative to the pipe buffer (NULL if the pipe buffer is empty)

The `ops` field points to the `anon_pipe_buf_ops` table of the pipe buffer's methods, which is a data structure of type `pipe_buf_operations`. Essentially, the table includes three methods:

map

Invoked before accessing data in the pipe buffer. It simply invokes `kmap ( )` on the pipe buffer's page frame, just in case the pipe buffer is stored in high memory (see the section "[Kernel Mappings of High-Memory Page Frames](#)" in [Chapter 8](#)).

unmap

Invoked when no longer accessing data in the pipe buffer. It invokes `kunmap ( )` on the pipe buffer's page frame.

release

Invoked when a pipe buffer is being released. The method implements a one-page memory cache: the page frame released is not the one storing the buffer, but a cached page frame pointed to by the `tmp_page` field of the `pipe_inode_info` data structure (if not `NULL`). The page frame that stored the buffer becomes the new cached page frame.

The 16 pipe buffers can be seen as a global, circular buffer: writing processes keep adding data to this large buffer, while reading process keep removing them. The number of bytes currently written in all pipe buffers and yet to be read is the so-called pipe size. For reasons of efficiency, the data yet to be read can be spread among several partially filled pipe buffers: in fact, each write operation may copy the data in a fresh, empty pipe buffer if the previous pipe buffer has not enough free space to store the new data. Hence, the kernel must keep track of:

- The pipe buffer that includes the next byte to be read, and the corresponding offset inside the page frame. The index of this pipe buffer is stored in the `curbuf` field of the `pipe_inode_info` data structure, while the offset is stored in the `offset` field of the corresponding `pipe_buffer` object.
- The first empty pipe buffer. Its index can be computed by adding (modulo 16) the index of the current pipe buffer, which is stored in the `curbuf` field of the `pipe_inode_info` data structure, and the number of pipe buffers with significant data, which is stored in the `nrbufs` field.

To avoid race conditions on the pipe's data structures, the kernel makes use of the `i_sem` semaphore included in the inode object.

19.1.2.1. The pipefs special filesystem

A pipe is implemented as a set of VFS objects, which have no corresponding disk images. In Linux 2.6, these VFS objects are organized into the pipefs special filesystem to expedite their handling (see the section "[Special Filesystems](#)" in [Chapter 12](#)). Because this filesystem has no mount point in the system directory tree, users never see it. However, thanks to pipefs, the pipes are fully integrated in the VFS layer, and the kernel can handle them in the same way as named pipes or FIFOs, which truly exist as files recognizable to end users (see the later section "[FIFOs](#)").

The `init_pipe_fs ( )` function, typically executed during kernel initialization, registers the pipefs filesystem and mounts it (refer to the discussion in the section "[Mounting a Generic Filesystem](#)" in [Chapter 12](#)):

```
struct file_system_type pipe_fs_type;
pipe_fs_type.name = "pipefs";
pipe_fs_type.get_sb = pipefs_get_sb;
pipe_fs.kill_sb = kill_anon_super;
```

```
register_filesystem(&pipe_fs_type);
pipe_mnt = do_kern_mount("pipefs", 0, "pipefs", NULL);
```

The mounted filesystem object that represents the root directory of pipefs is stored in the `pipe_mnt` variable.

19.1.3. Creating and Destroying a Pipe

The `pipe( )` system call is serviced by the `sys_pipe( )` function, which in turn invokes the `do_pipe( )` function. To create a new pipe, `do_pipe( )` performs the following operations:

1. Invokes the `get_pipe_inode( )` function, which allocates and initializes an inode object for the pipe in the pipefs filesystem. In particular, this function executes the following actions:
 - a. Allocates a new inode in the pipefs filesystem.
 - b. Allocates a `pipe_inode_info` data structure and stores its address in the `i_pipe` field of the inode.
 - c. Sets the `curbuf` and `nrbufs` fields of the `pipe_inode_info` structure to 0; also, fills with zeros all fields of the pipe buffer objects in the `bufs` array.
 - d. Initializes the `r_counter` and `w_counter` fields of the `pipe_inode_info` structure to 1.
 - e. Sets the `readers` and `writers` fields of the `pipe_inode_info` structure to 1.
2. Allocates a file object and a file descriptor for the read channel of the pipe, sets the `f_flag` field of the file object to `O_RDONLY`, and initializes the `f_op` field with the address of the `read_pipe_fops` table.
3. Allocates a file object and a file descriptor for the write channel of the pipe, sets the `flag` field of the file object to `O_WRONLY`, and initializes the `f_op` field with the address of the `write_pipe_fops` table.
4. Allocates a dentry object and uses it to link the two file objects and the inode object (see the section "[The Common File Model](#)" in [Chapter 12](#)); then inserts the new inode in the pipefs special filesystem.
5. Returns the two file descriptors to the User Mode process.

The process that issues a `pipe( )` system call is initially the only process that can access the new pipe, both for reading and writing. To represent that the pipe has both a reader and a writer, the `readers` and `writers` fields of the `pipe_inode_info` data structure are initialized to 1. In general, each of these two fields is set to 1 only if the corresponding pipe's file object is still opened by a process; the field is set to 0 if the corresponding file object has been released, because it is no longer accessed by any process.

Forking a new process does not increase the value of the `readers` and `writers` fields, so they never rise above 1;^[*] however, it does increase the value of the usage counters of all file objects still used by the parent process (see the section "[The clone\(\), fork\(\), and vfork\(\) System Calls](#)" in [Chapter 3](#)). Thus, the objects are not released even when the parent dies, and the pipe stays open for use by the children.

[*] As we'll see, the `readers` and `writers` fields act as counters instead of flags when associated with FIFOs.

Whenever a process invokes the `close( )` system call on a file descriptor associated with a pipe, the kernel executes the `fput( )` function on the corresponding file object, which decreases the usage counter. If the counter becomes 0, the function invokes the `release` method of the file operations (see the sections "[The close\(\) System Call](#)" and "[Files Associated with a Process](#)" in [Chapter 12](#)).

Depending on whether the file is associated with the read or write channel, the `release` method is implemented by either `pipe_read_release( )` or `pipe_write_release( )`; both functions invoke `pipe_release( )`, which sets either the `readers` field or the `writers` field of the `pipe_inode_info` structure to 0. The function checks whether both the `readers` and `writers` fields are equal to 0; in this case, it invokes the pipe buffer's `release` method of all pipe buffers, thus releasing to the buddy system all pipe's page frames; moreover, the function releases the cached page frame pointed to by the `tmp_page` field. Otherwise, if either the `readers` field or the `writers` field is not zero, the function wakes up the processes sleeping in the pipe's wait queue so they can recognize the change in the pipe state.

19.1.4. Reading from a Pipe

A process wishing to get data from a pipe issues a `read( )` system call, specifying the file descriptor associated with the pipe's reading end. As described in the section "[The read\(\) and write\(\) System Calls](#)" in [Chapter 12](#), the kernel ends up invoking the `read` method found in the file operation table associated with the proper file object. In the case of a pipe, the entry for the `read` method in the `read_pipe_fops` table points to the `pipe_read( )` function.

The `pipe_read( )` function is quite involved, because the POSIX standard specifies several requirements for the pipe's read operations. [Table 19-3](#) summarizes the expected behavior of a `read( )` system call that requests `n` bytes from a pipe that has a pipe size (number of bytes in the pipe buffers yet to be read) equal to `p`.

The system call might block the current process in two cases:

- The pipe buffer is empty when the system call starts.
- The pipe buffer does not include all requested bytes, and a writing process was previously put to sleep while waiting for space in the buffer.

Notice that the read operation can be nonblocking: in this case, it completes as soon as all available bytes (even none) are copied into the user address space.^[*]

[*] Nonblocking operations are usually requested by specifying the `O_NONBLOCK` flag in the `open( )` system call. This method does not work for pipes, because they cannot be opened. A process can, however, require a nonblocking operation on a pipe by issuing a `fcntl( )` system call on the corresponding file descriptor.

Notice also that the value 0 is returned by the `read( )` system call only if the pipe is empty and no process is currently using the file object associated with the pipe's write channel.

Table 19-3. Reading `n` bytes from a pipe

At least one writing process

No writing process

Blocking read

Nonblocking read

Pipe Size p

Sleeping writer

No sleeping writer

$p = 0$

Copy n bytes and return n , waiting for data when the pipe buffer is empty.

Wait for some data, copy it, and return its size.

Return

`-EAGAIN`.

Return 0.

$0 < p < n$

Copy p bytes and return p : 0 bytes are left in the pipe buffer.

$p \geq n$

Copy n bytes and return n : $p-n$ bytes are left in the pipe buffer.

The function performs the following operations:

1. Acquires the `i_sem` semaphore of the inode.
2. Determines whether the pipe size is 0 by reading the `nrbufs` field of the `pipe_inode_info` structure; if the field is equal to zero, all pipe buffers are empty. In this case, it determines whether the function must return or whether the process must be blocked while waiting until another process writes some data in the pipe (see [Table 19-3](#)). The type of I/O operation (blocking or nonblocking) is specified by the `O_NONBLOCK` flag in the `f_flags` field of the file object. If the current process must be blocked, the function performs the following actions:
 - a. Invokes `prepare_to_wait( )` to add `current` to the wait queue of the pipe (the `wait` field of the `pipe_inode_info` structure).
 - b. Releases the inode semaphore.
 - c. Invokes `schedule( )`.
 - d. Once awake, invokes `finish_wait( )` to remove `current` from the wait queue, acquires again the `i_sem` inode semaphore, and then jumps back to step 2.
3. Gets the index of the current pipe buffer from the `curbuf` field of the `pipe_inode_info` data structure.
4. Executes the `map` method of the pipe buffer.
5. Copies the requested number of bytes or the number of available bytes in the pipe buffer, if it is

- smaller from the pipe's buffer to the user address space.
6. Executes the `unmap` method of the pipe buffer.
 7. Updates the `offset` and `len` fields of the corresponding `pipe_buffer` object.
 8. If the pipe buffer has been emptied (`len` fields of the `pipe_buffer` object now equal to zero), it invokes the pipe buffer's `release` method to free the corresponding page frame, sets the `ops` field in the `pipe_buffer` object to `NULL`, advances the index of the current pipe buffer stored in the `curbuf` field of the `pipe_inode_info` data structure, and decreases the counter of nonempty pipe buffers in the `nrbufs` field.
 9. If all requested bytes have been copied, it jumps to step 12.
 10. Here not all requested bytes have been copied to the User Mode address space. If the pipe size is greater than zero (`nrbufs` field of the `pipe_inode_info` data structure not null), it goes back to step 3.
 11. There are no more bytes left in the pipe buffers. If there is at least one writing process currently sleeping (that is, the `waiting_writers` field of the `pipe_inode_info` data structure is greater than 0), and the read operation is blocking, it invokes `wake_up_interruptible_sync( )` to wake up all processes sleeping on the pipe's wait queue, and jumps back to step 2.
 12. Releases the `i_sem` semaphore of the inode.
 13. Invokes `wake_up_interruptible_sync( )` to wake up all writer processes sleeping on the pipe's wait queue.
 14. Returns the number of bytes copied into the user address space.

19.1.5. Writing into a Pipe

A process wishing to put data into a pipe issues a `write( )` system call, specifying the file descriptor for the writing end of the pipe. The kernel satisfies this request by invoking the `write` method of the proper file object; the corresponding entry in the `write_pipe_fops` table points to the `pipe_write( )` function.

Table 19-4 summarizes the behavior, specified by the POSIX standard, of a `write( )` system call that requested to write `n` bytes into a pipe having `u` unused bytes in its buffer. In particular, the standard requires that write operations involving a small number of bytes must be atomically executed. More precisely, if two or more processes are concurrently writing into a pipe, each write operation involving fewer than 4,096 bytes (the pipe buffer size) must finish without being interleaved with write operations of other processes to the same pipe. However, write operations involving more than 4,096 bytes may be nonatomic and may also force the calling process to sleep.

Table 19-4. Writing `n` bytes to a pipe

At least one reading process

Available buffer space `u`

Blocking write

Nonblocking write

No reading process

$u < n \leq 4,096$

Wait until $n-u$ bytes are freed, copy n bytes, and return n .

Return `-EAGAIN`.

Send `SIGPIPE` signal and return `-EPIPE`.

$n > 4,096$

Copy n bytes (waiting when necessary) and return n .

If $u > 0$, copy u bytes and return u ; return `-EAGAIN`.

$u \geq n$

Copy n bytes and return n .

Moreover, each write operation to a pipe must fail if the pipe does not have a reading process (that is, if the `readers` field of the pipe's inode object has the value 0). In this case, the kernel sends a `SIGPIPE` signal to the writing process and terminates the `write( )` system call with the `-EPIPE` error code, which usually leads to the familiar "Broken pipe" message.

The `pipe_write( )` function performs the following operations:

1. Acquires the `i_sem` semaphore of the inode.
2. Checks whether the pipe has at least one reading process. If not, it sends a `SIGPIPE` signal to the current process, releases the inode semaphore, and returns an `-EPIPE` value.
3. Determines the index of the last written pipe buffers by adding the `curbuf` and `nrbufs` fields of the `pipe_inode_info` data structure and subtracting 1. If this pipe buffer has enough free space to store all the bytes to be written, then it copies the data into it:
 - a. Executes the `map` method of the pipe buffer.
 - b. Copies all the bytes in the pipe buffer.
 - c. Executes the `unmap` method of the pipe buffer.
 - d. Updates the `len` field of the corresponding `pipe_buffer` object.
 - e. Jumps to step 11.
4. If the `nrbufs` field of the `pipe_inode_info` data structure is equal to 16, there is no empty pipe buffer to store the bytes (yet) to be written. In this case:
 - a. If the write operation is nonblocking, it jumps to step 11 to terminate by returning the `-EAGAIN` error code.
 - b. If the write operation is blocking, it adds 1 to the `waiting_writers` field of the `pipe_inode_info` structure, invokes `prepare_to_wait( )` to add current to the wait queue of the pipe (the `wait` field of the `pipe_inode_info` structure), releases the inode semaphore, and invokes `schedule( )`. Once awake, it invokes `finish_wait( )` to remove current from the wait queue, again acquires the inode semaphore, decreases the `waiting_writers` field, and then jumps back to step 4.
5. Now there is at least one empty pipe buffer. Determines the index of the first empty pipe buffer by adding the `curbuf` and `nrbufs` fields of the `pipe_inode_info` data structure.

6. Allocates a new page frame from the buddy system, unless the `tmp_page` field of the `pipe_inode_info` data structure is not `NULL`.
7. Copies up to 4,096 bytes from the User Mode address space into the page frame (temporarily mapping it in the Kernel Mode linear address space, if necessary).
8. Updates the fields of the `pipe_buffer` object associated with the pipe buffer by setting the `page` field to the address of the page frame descriptor, the `ops` field to the address of the `anon_pipe_buf_ops` table, the `offset` field to 0, and the `len` field to the number of written bytes.
9. Increases the counter of nonempty pipe buffers stored in the `nrbufs` field of the `pipe_inode_info` data structure.
10. If not all requested bytes were written, it jumps back to step 4.
11. Releases the inode semaphore.
12. Wakes up all reader processes sleeping on the pipe's wait queue.
13. Returns the number of bytes written into the pipe's buffer (or an error code if writing was not possible).

19.2. FIFOs

Although pipes are a simple, flexible, and efficient communication mechanism, they have one main drawbacknamely, that there is no way to open an already existing pipe. This makes it impossible for two arbitrary processes to share the same pipe, unless the pipe was created by a common ancestor process.

This drawback is substantial for many application programs. Consider, for instance, a database engine server, which continuously polls client processes wishing to issue some queries and which sends the results of the database lookups back to them. Each interaction between the server and a given client might be handled by a pipe. However, client processes are usually created on demand by a command shell when a user explicitly queries the database; server and client processes thus cannot easily share a pipe.

To address such limitations, Unix systems introduce a special file type called a named pipe or FIFO (which stands for "first in, first out;" the first byte written into the special file is also the first byte that is read). Each FIFO is much like a pipe: rather than owning disk blocks in the filesystems, an opened FIFO is associated with a kernel buffer that temporarily stores the data exchanged by two or more processes.

Thanks to the disk inode, however, a FIFO can be accessed by every process, because the FIFO filename is included in the system's directory tree. Thus, in our example, the communication between server and clients may be easily established by using FIFOs instead of pipes. The server creates, at startup, a FIFO used by client programs to make their requests. Each client program creates, before establishing the connection, another FIFO to which the server program can write the answer to the query and includes the FIFO's name in the initial request to the server.

In Linux 2.6, FIFOs and pipes are almost identical and use the same `pipe_inode_info` structures. As a matter of fact, the `read` and `write` file operation methods of a FIFO are implemented by the same `pipe_read( )` and `pipe_write( )` functions described in the earlier sections "[Reading from a Pipe](#)" and "[Writing into a Pipe](#)." Actually, there are only two significant differences:

- FIFO inodes appear on the system directory tree rather than on the pipefs special

filesystem.

- FIFOs are a bidirectional communication channel; that is, it is possible to open a FIFO in read/write mode.

To complete our description, therefore, we just have to explain how FIFOs are created and opened.

19.2.1. Creating and Opening a FIFO

A process creates a FIFO by issuing a `mknod( )` ^[*] system call (see the section "[Device Files](#)" in [Chapter 13](#)), passing to it as parameters the pathname of the new FIFO and the value `S_IFIFO (0x10000)` logically ORed with the permission bit mask of the new file. POSIX introduces a function named `mkfifo( )` specifically to create a FIFO. This call is implemented in Linux, as in System V Release 4, as a C library function that invokes `mknod( )`.

^[*] In fact, `mknod( )` can be used to create nearly every kind of file, such as block and character device files, FIFOs, and even regular files (it cannot create directories or sockets, though).

Once created, a FIFO can be accessed through the usual `open( )`, `read( )`, `write( )`, and `close( )` system calls, but the VFS handles it in a special way, because the FIFO inode and file operations are customized and do not depend on the filesystems in which the FIFO is stored.

The POSIX standard specifies the behavior of the `open( )` system call on FIFOs; the behavior depends essentially on the requested access type, the kind of I/O operation (blocking or nonblocking), and the presence of other processes accessing the FIFO.

A process may open a FIFO for reading, for writing, or for reading and writing. The file operations associated with the corresponding file object are set to special methods for these three cases.

When a process opens a FIFO, the VFS performs the same operations as it does for device files (see the section "[VFS Handling of Device Files](#)" in [Chapter 13](#)). The inode object associated with the opened FIFO is initialized by a filesystem-dependent `read_inode` superblock method; this method

always checks whether the inode on disk represents a special file, and invokes, if necessary, the `init_special_inode( )` function. It turn, this function sets the `i_fop` field of the inode object to the address of the `def_fifo_fops` table. Later, the kernel sets the file operation table of the file object to `def_fifo_fops`, and executes its `open` method, which is implemented by `fifo_open( )`.

The `fifo_open( )` function initializes the data structures specific to the FIFO; in particular, it performs the following operations:

1. Acquires the `i_sem` inode semaphore.
2. Checks the `i_pipe` field of the inode object; if it is `NULL`, it allocates and it initializes a new `pipe_inode_info` structure, as in steps 1b-1e in the earlier section "[Creating and Destroying a Pipe](#)."
3. Depending on the access mode specified as the parameter of the `open( )` system call, it initializes the `f_op` field of the file object with the address of the proper file operation table (see [Table 19-5](#)).

Table 19-5. FIFO's file operations

Access type	File operations	Read method	Write method
Read-only	<code>read_fifo_fops</code>	<code>pipe_read()</code>	<code>bad_pipe_w()</code>
Write-only	<code>write_fifo_fops</code>	<code>bad_pipe_r()</code>	<code>pipe_write()</code>
Read/write	<code>rdwr_fifo_fops</code>	<code>pipe_read()</code>	<code>pipe_write()</code>

- If the access mode is either read-only or read/write, it adds one to the `readers` and `r_counter` fields of the `pipe_inode_info` structure. Moreover, if the access mode is read-only and there is no other reading process, it wakes up any writing process sleeping in the wait queue.
- If the access mode is either write-only or read/write, it adds one to the `writers` and `w_counter` fields of the `pipe_inode_info` structure. Moreover, if the access mode is write-only and there is no other writing process, it wakes up any reading process sleeping in the wait queue.
- If there are no readers or no writers, it decides whether the function should block or terminate returning an error code (see [Table 19-6](#)).

Table 19-6. Behavior of the `fifo_open( )` function

Access type

Blocking

Nonblocking

Read-only, with writers

Successfully return

Successfully return

Read-only, no writer

Wait for a writer

Successfully return

Write-only, with readers

Successfully return

Successfully return

Write-only, no reader

Wait for a reader

Return `-ENXIO`

Read/write

Successfully return

Successfully return

- Releases the inode semaphore, and terminates, returning 0 (success).

The FIFO's three specialized file operation tables differ mainly in the implementation of the `read` and `write` methods. If the access type allows read operations, the `read` method is implemented by the `pipe_read( )` function. Otherwise, it is implemented by `bad_pipe_r( )`, which only returns an error code. Similarly, if the access type allows write operations, the `write` method is implemented by the `pipe_write( )` function; otherwise, it is implemented by `bad_pipe_w( )`, which also returns an error code.

19.3. System V IPC

IPC is an abbreviation for Interprocess Communication and commonly refers to a set of mechanisms that allow a User Mode process to do the following:

- Synchronize itself with other processes by means of semaphores
- Send messages to other processes or receive messages from them
- Share a memory area with other processes

System V IPC first appeared in a development Unix variant called "Columbus Unix " and later was adopted by AT&T's System III . It is now found in most Unix systems, including Linux.

IPC data structures are created dynamically when a process requests an IPC resource (a semaphore, a message queue, or a shared memory region). An IPC resource is persistent: unless explicitly removed by a process, it is kept in memory and remains available until the system is shut down. An IPC resource may be used by every process, including those that do not share the ancestor that created the resource.

Because a process may require several IPC resources of the same type, each new resource is identified by a 32-bit IPC key, which is similar to the file pathname in the system's directory tree. Each IPC resource also has a 32-bit IPC identifier, which is somewhat similar to the file descriptor associated with an open file. IPC identifiers are assigned to IPC resources by the kernel and are unique within the system, while IPC keys can be freely chosen by programmers.

When two or more processes wish to communicate through an IPC resource, they all refer to the IPC identifier of the resource.

19.3.1. Using an IPC Resource

IPC resources are created by invoking the `semget ( )`, `msgget ( )`, or `shmget ( )` functions, depending on whether the new resource is a semaphore, a message queue, or a shared memory region.

The main objective of each of these three functions is to derive from the IPC key (passed as the first parameter) the corresponding IPC identifier, which is then used by the process for accessing the resource. If there is no IPC resource already associated with the IPC key, a new resource is created. If everything goes right, the function returns a positive IPC identifier; otherwise, it returns one of the error codes listed in [Table 19-7](#).

Table 19-7. Error codes returned while requesting an IPC identifier

Error code	Description
EACCESS	Process does not have proper access rights
EEXIST	Process tried to create an IPC resource with the same key as one that already exists
EINVAL	Invalid argument in a parameter of <code>semget ()</code> , <code>msgget ()</code> , or <code>shmget ()</code>
ENOENT	No IPC resource with the requested key exists and the process did not ask to create it
ENOMEM	No more storage is left for an additional IPC resource
ENOSPC	Maximum limit on the number of IPC resources has been exceeded

Assume that two independent processes want to share a common IPC resource. This can be achieved in two possible ways:

- The processes agree on some fixed, predefined IPC key. This is the simplest case, and it works quite well for every complex application implemented by many processes. However, there's a chance that the same IPC key is chosen by another unrelated program. In this case, the IPC functions might be successfully invoked and still return the IPC identifier of the wrong resource.^[*]

^[*] The `ftok ( )` function attempts to create a new key from a file pathname and an 8-bit project identifier passed as its parameters. It does not guarantee, however, a unique key number, because there is a small chance that it will return the same IPC key to two different applications using different pathnames and project identifiers.

- One process issues a `semget ( )`, `msgget ( )`, or `shmget ( )` function by specifying `IPC_PRIVATE` as its IPC key. A new IPC resource is thus allocated, and the process can either communicate its IPC identifier to the other process in the application^[†] or fork the other process itself. This method ensures that the IPC resource cannot be used accidentally by other applications.

^[†] This implies, of course, the existence of another communication channel between the processes not based on IPC.

The last parameter of the `semget ( )`, `msgget ( )`, and `shmget ( )` functions can include three flags. `IPC_CREAT` specifies that the IPC resource must be created, if it does not already exist; `IPC_EXCL` specifies that the function must fail if the resource already exists and the `IPC_CREAT` flag is set; `IPC_NOWAIT` specifies that the process should never block when accessing the IPC resource (typically, when fetching a message or when acquiring a semaphore).

Even if the process uses the `IPC_CREAT` and `IPC_EXCL` flags, there is no way to ensure exclusive access to an IPC resource, because other processes may always refer to the resource by using its IPC identifier.

To minimize the risk of incorrectly referencing the wrong resource, the kernel does not recycle IPC identifiers as soon as they become free. Instead, the IPC identifier assigned to a resource is almost always larger than the identifier assigned to the previously allocated resource of the same type. (The only exception occurs when the 32-bit IPC identifier overflows.) Each IPC identifier is computed by combining a slot usage sequence number relative to the resource type, an arbitrary slot index for the allocated resource, and an arbitrary value chosen in the kernel that is greater than the maximum number of allocatable resources. If we choose s to represent the slot usage sequence number, M to represent the upper bound on the number of allocatable resources, and i to represent the slot index, where $0 \leq i < M$, each IPC resource's ID is computed as follows:

$$\text{IPC identifier} = s \times M + i$$

In Linux 2.6, the value of M is set to 32,768 (`IPCMNI` macro). The slot usage sequence number s is initialized to 0 and is increased by 1 at every resource allocation. When s reaches a predefined threshold, which depends on the type of IPC resource, it restarts from 0.

Every type of IPC resource (semaphores, message queues, and shared memory areas) owns an `ipc_ids` data structure, which includes the fields shown in [Table 19-8](#).

Table 19-8. The fields of the `ipc_ids` data structure

Type	
Field	
Description	
<code>int</code>	
<code>in_use</code>	
Number of allocated IPC resources	
<code>int</code>	
<code>max_id</code>	
Maximum slot index in use	
<code>unsigned short</code>	
<code>seq</code>	
Slot usage sequence number for the next allocation	
<code>unsigned short</code>	
<code>seq_max</code>	
Maximum slot usage sequence number	
<code>struct semaphore</code>	
<code>sem</code>	

Semaphore protecting the `ipc_ids` data structure

```
struct ipc_id_ary
```

```
nullentry
```

Fake data structure pointed to by the `entries` field if this IPC resource cannot be initialized (normally not used)

```
struct ipc_id_ary *
```

```
enTRies
```

Pointer to the `ipc_id_ary` data structure for this resource

The `ipc_id_ary` data structure consists of two fields: `p` and `size`. The `p` field is an array of pointers to `kern_ipc_perm` data structures, one for every allocatable resource. The `size` field is the size of this array. Initially, the array stores 1, 16, or 128 pointers, respectively for shared memory regions, message queues, and semaphores. The kernel dynamically increases the size of the array when it becomes too small. However, there is an upper bound on the number of resources for each given type. The system administrator may change these bounds by writing into the `/proc/sys/kernel/sem`, `/proc/sys/kernel/msgmni`, and `/proc/sys/kernel/shmmni` files, respectively.

Each `kern_ipc_perm` data structure is associated with an IPC resource and contains the fields shown in [Table 19-9](#). The `uid`, `gid`, `cuid`, and `cgid` fields store the user and group identifiers of the resource's creator and the user and group identifiers of the current resource's owner, respectively. The `mode` bit mask includes six flags, which store the read and write access permissions for the resource's owner, the resource's group, and all other users. IPC access permissions are similar to file access permissions described in the section "[Access Rights and File Mode](#)" in [Chapter 1](#), except that the Execute permission flag is not used.

Table 19-9. The fields in the `kern_ipc_perm` structure

Type

Field

Description

```
spinlock_t
```

```
lock
```

Spin lock protecting the IPC resource descriptor

```
int
```

```
deleted
```

Flag set if the resource has been released

```
int
```

key

IPC key

unsigned int

uid

Owner user ID

unsigned int

gid

Owner group ID

unsigned int

cuid

Creator user ID

unsigned int

cgid

Creator group ID

unsigned short

mode

Permission bit mask

unsigned long

seq

Slot usage sequence number

void *

security

Pointer to a security structure (used by SELinux)

The `kern_ipc_perm` data structure also includes a `key` field (which contains the IPC key of the corresponding resource) and a `seq` field (which stores the slot usage sequence number s used to compute the IPC identifier of the resource).

The `semctl( )`, `msgctl( )`, and `shmctl( )` functions may be used to handle IPC resources. The `IPC_SET` subcommand allows a process to change the owner's user and group identifiers and the permission

bit mask in the `ipc_perm` data structure. The `IPC_STAT` and `IPC_INFO` subcommands retrieve some information concerning a resource. Finally, the `IPC_RMID` subcommand releases an IPC resource. Depending on the type of IPC resource, other specialized subcommands are also available.^[*]

[*] An IPC design flaw is that a User Mode process cannot atomically create and initialize an IPC semaphore, because these two operations are performed by two different IPC functions.

Once an IPC resource is created, a process may act on the resource by means of a few specialized functions. A process may acquire or release an IPC semaphore by issuing the `semop ( )` function. When a process wants to send or receive an IPC message, it uses the `msgsnd ( )` and `msgrcv ( )` functions, respectively. Finally, a process attaches and detaches an IPC shared memory region in its address space by means of the `shmat ( )` and `shmdt ( )` functions, respectively.

19.3.2. The `ipc ( )` System Call

All IPC functions must be implemented through suitable Linux system calls. Actually, in the 80 x 86 architecture, there is just one IPC system call named `ipc ( )`. When a process invokes an IPC function, let's say `msgget ( )`, it really invokes a wrapper function in the C library. This in turn invokes the `ipc ( )` system call by passing to it all the parameters of `msgget ( )` plus a proper subcommand code in this case, `MSGGET`. The `sys_ipc ( )` service routine examines the subcommand code and invokes the kernel function that implements the requested service.

The `ipc ( )` "multiplexer" system call is a legacy from older Linux versions, which included the IPC code in a dynamic module (see [Appendix B](#)). It did not make much sense to reserve several system call entries in the `system_call` table for a kernel component that could be missing, so the kernel designers adopted the multiplexer approach.

Nowadays, System V IPC can no longer be compiled as a dynamic module, and there is no justification for using a single IPC system call. As a matter of fact, Linux provides one system call for each IPC function on Hewlett-Packard's Alpha architecture and on Intel's IA-64.

19.3.3. IPC Semaphores

IPC semaphores are quite similar to the kernel semaphores introduced in [Chapter 5](#); they are counters used to provide controlled access to shared data structures for multiple processes.

The semaphore value is positive if the protected resource is available, and 0 if the protected resource is currently not available. A process that wants to access the resource tries to decrease the semaphore value; the kernel, however, blocks the process until the operation on the semaphore yields a positive value. When a process relinquishes a protected resource, it increases its semaphore value; in doing so, any other process waiting for the semaphore is woken up.

Actually, IPC semaphores are more complicated to handle than kernel semaphores for two main reasons:

- Each IPC semaphore is a set of one or more semaphore values, not just a single value like a kernel semaphore. This means that the same IPC resource can protect several independent shared data structures. The number of semaphore values in each IPC semaphore must be specified as a parameter of the `semget ( )` function when the resource is being allocated. From now on, we'll refer to the

counters inside an IPC semaphore as primitive semaphores. There are bounds both on the number of IPC semaphore resources (by default, 128) and on the number of primitive semaphores inside a single IPC semaphore resource (by default, 250); however, the system administrator can easily modify these bounds by writing into the `/proc/sys/kernel/sem` file.

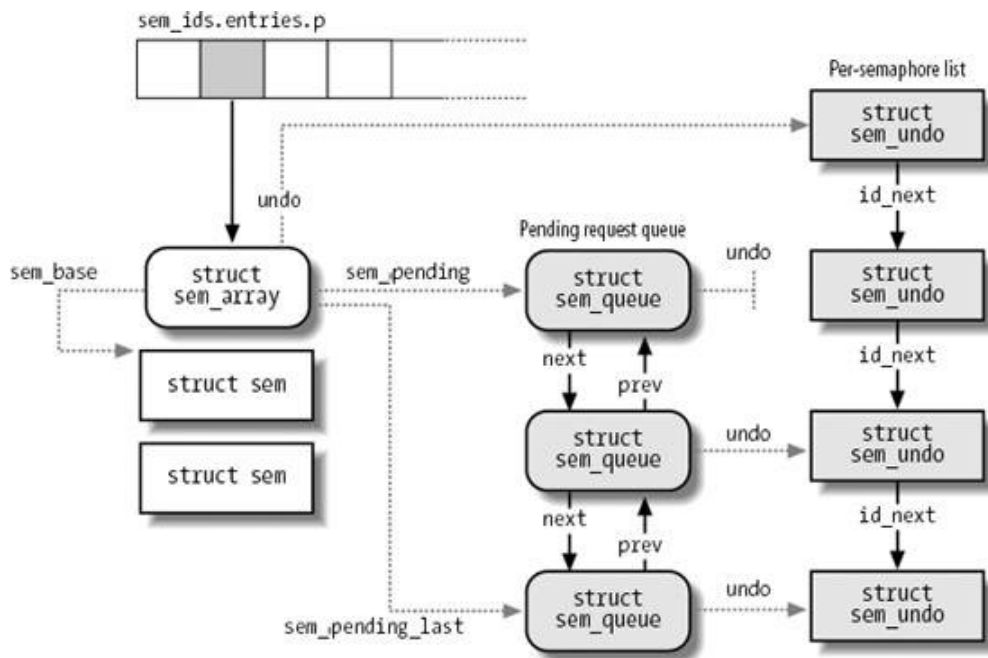
- System V IPC semaphores provide a fail-safe mechanism for situations in which a process dies without being able to undo the operations that it previously issued on a semaphore. When a process chooses to use this mechanism, the resulting operations are called undoable semaphore operations. When the process dies, all of its IPC semaphores can revert to the values they would have had if the process had never started its operations. This can help prevent other processes that use the same semaphores from remaining blocked indefinitely as a consequence of the terminating process failing to manually undo its semaphore operations.

First, we'll briefly sketch the typical steps performed by a process wishing to access one or more resources protected by an IPC semaphore:

1. Invokes the `semget ( )` wrapper function to get the IPC semaphore identifier, specifying as the parameter the IPC key of the IPC semaphore that protects the shared resources. If the process wants to create a new IPC semaphore, it also specifies the `IPC_CREATE` or `IPC_PRIVATE` flag and the number of primitive semaphores required (see the section "[Using an IPC Resource](#)" earlier in this chapter).
2. Invokes the `semop ( )` wrapper function to test and decrease all primitive semaphore values involved. If all the tests succeed, the decrements are performed, the function terminates, and the process is allowed to access the protected resources. If some semaphores are in use, the process is usually suspended until some other process releases the resources. The function receives as its parameters the IPC semaphore identifier, an array of integers specifying the operations to be atomically performed on the primitive semaphores, and the number of such operations. Optionally, the process may specify the `SEM_UNDO` flag, which instructs the kernel to reverse the operations, should the process exit without releasing the primitive semaphores.
3. When relinquishing the protected resources, it invokes the `semop ( )` function again to atomically increase all primitive semaphores involved.
4. Optionally, it invokes the `semctl ( )` wrapper function, specifying the `IPC_RMID` command to remove the IPC semaphore from the system.

Now we can discuss how the kernel implements IPC semaphores. The data structures involved are shown in [Figure 19-1](#). The `sem_ids` variable stores the `ipc_ids` data structure of the IPC semaphore resource type; the corresponding `ipc_id_ary` data structure contains an array of pointers to `sem_array` data structures, one item for every IPC semaphore resource.

Figure 19-1. IPC semaphore data structures



Formally, the array stores pointers to `kern_ipc_perm` data structures, but each structure is simply the first field of the `sem_array` data structure. All fields of the `sem_array` data structure are shown in [Table 19-10](#).

Table 19-10. The fields in the `sem_array` data structure

Type

Field

Description

`struct kern_ipc_perm`

`sem_perm`

`kern_ipc_perm` data structure

`long`

`sem_otime`

Timestamp of last `semop()`

`long`

`sem_ctime`

Timestamp of last change

`struct sem *`

`sem_base`

Pointer to first `sem` structure

`struct sem_queue *`

`sem_pending`

Pending operations

`struct sem_queue **`

`sem_pending_last`

Last pending operation

`struct sem_undo *`

`undo`

Undo requests

`unsigned long`

`sem_nsems`

Number of semaphores in array

The `sem_base` field points to an array of `sem` data structures, one for every IPC primitive semaphore. The latter data structure includes only two fields:

`semval`

The value of the semaphore's counter.

`sempid`

The PID of the last process that accessed the semaphore. This value can be queried by a process through the `semctl( )` wrapper function.

19.3.3.1. Undoable semaphore operations

If a process aborts suddenly, it cannot undo the operations that it started (for instance, release the semaphores it reserved); so by declaring them undoable, the process lets the kernel return the semaphores to a consistent state and allow other processes to proceed. Processes can request undoable operations by specifying the `SEM_UNDO` flag in the `semop( )` function.

Information to help the kernel reverse the undoable operations performed by a given process on a given IPC semaphore resource is stored in a `sem_undo` data structure. It essentially contains the IPC identifier of the

semaphore and an array of integers representing the changes to the primitive semaphore's values caused by all undoable operations performed by the process.

A simple example can illustrate how such `sem_undo` elements are used. Consider a process that uses an IPC semaphore resource containing four primitive semaphores. Suppose that it invokes the `semop( )` function to increase the first counter by 1 and decrease the second by 2. If it specifies the `SEM_UNDO` flag, the integer in the first array element in the `sem_undo` data structure is decreased by 1, the integer in the second element is increased by 2, and the other two integers are left unchanged. Further undoable operations on the IPC semaphore performed by the same process change the integers stored in the `sem_undo` structure accordingly. When the process exits, any nonzero value in that array corresponds to one or more unbalanced operations on the corresponding primitive semaphore; the kernel reverses these operations, simply adding the nonzero value to the corresponding semaphore's counter. In other words, the changes made by the aborted process are backed out while the changes made by other processes are still reflected in the state of the semaphores.

For each process, the kernel keeps track of all semaphore resources handled with undoable operations so that it can roll them back if the process unexpectedly exits. Furthermore, for each semaphore, the kernel has to keep track of all its `sem_undo` structures so it can quickly access them whenever a process uses `semctl( )` to force an explicit value into a primitive semaphore's counter or to destroy an IPC semaphore resource.

The kernel is able to handle these tasks efficiently, thanks to two lists, which we denote as the per-process and the per-semaphore lists. The first list keeps track of all semaphores operated upon by a given process with undoable operations. The second list keeps track of all processes that are acting on a given semaphore with undoable operations. More precisely:

- The per-process list includes all `sem_undo` data structures corresponding to IPC semaphores on which the process has performed undoable operations. The `sysvsem.undo_list` field of the process descriptor points to a data structure, of type `sem_undo_list`, which in turn contains a pointer to the first element of the list; the `proc_next` field of each `sem_undo` data structure points to the next element in the list. (As mentioned in the section "[The clone\(\), fork\(\), and vfork\(\) System Calls](#)" in [Chapter 3](#), clone processes created by passing the `CLONE_SYSVSEM` flag to the `clone( )` system call share the same list of undoable semaphore operations, because they share the same `sem_undo_list` descriptor.)
- The per-semaphore list includes all `sem_undo` data structures corresponding to the processes that performed undoable operations on the semaphore. The `undo` field of the `sem_array` data structure points to the first element of the list, while the `id_next` field of each `sem_undo` data structure points to the next element in the list.

The per-process list is used when a process terminates. The `exit_sem( )` function, which is invoked by `do_exit( )`, walks through the list and reverses the effect of any unbalanced operation for every IPC semaphore touched by the process. By contrast, the per-semaphore list is mainly used when a process invokes the `semctl( )` function to force an explicit value into a primitive semaphore. The kernel sets the corresponding element to 0 in the arrays of all `sem_undo` data structures referring to that IPC semaphore resource, because it would no longer make any sense to reverse the effect of previous undoable operations performed on that primitive semaphore. Moreover, the per-semaphore list is also used when an IPC semaphore is destroyed; all related `sem_undo` data structures are invalidated by setting the `semid` field to `-1`.^[*]

[*] Notice that they are just invalidated and not freed, because it would be too costly to remove the data structures from the per-process lists of all processes.

19.3.3.2. The queue of pending requests

The kernel associates a queue of pending requests with each IPC semaphore to identify processes that are waiting on one (or more) of the semaphores in the array. The queue is a doubly linked list of `sem_queue` data structures whose fields are shown in [Table 19-11](#). The first and last pending requests in the queue are referenced, respectively, by the `sem_pending` and `sem_pending_last` fields of the `sem_array` structure. This last field allows the list to be handled as easily as a FIFO; new pending requests are added to the end of the list so they will be serviced later. The most important fields of a pending request are `nsops` (which stores the number of primitive semaphores involved in the pending operation) and `sops` (which points to an array of integer values describing each semaphore operation). The `sleeper` field stores the descriptor address of the sleeping process that requested the operation.

Table 19-11. The fields in the `sem_queue` data structure

Type
Field
Description
<code>struct sem_queue *</code>
<code>next</code>
Pointer to next queue element
<code>struct sem_queue **</code>
<code>prev</code>
Pointer to previous queue element
<code>struct task_struct *</code>
<code>sleeper</code>
Pointer to the sleeping process that requested the semaphore operation
<code>struct sem_undo *</code>
<code>undo</code>
Pointer to <code>sem_undo</code> structure
<code>int</code>
<code>pid</code>
Process identifier
<code>int</code>

status

Completion status of operation

struct sem_array *

sma

Pointer to IPC semaphore descriptor

int

id

Slot index of the IPC semaphore resource

struct sembuf *

sops

Pointer to array of pending operations

int

nsops

Number of pending operations

int

alter

Flag denoting whether the operation modifies the semaphore array

Figure 19-1 illustrates an IPC semaphore that has three pending requests. The second and third requests refer to undoable operations, so the `undo` field of the `sem_queue` data structure points to the corresponding `sem_undo` structure; the first pending request has a `NULL` `undo` field because the corresponding operation is not undoable.

19.3.4. IPC Messages

Processes can communicate with one another by means of IPC messages . Each message generated by a process is sent to an IPC message queue, where it stays until another process reads it.

A message is composed of a fixed-size header and a variable-length text; it can be labeled with an integer value (the message type), which allows a process to selectively retrieve messages from its message queue.^[*] Once a process has read a message from an IPC message queue, the kernel destroys the message; therefore, only one process can receive a given message.

[*] As we'll see, the message queue is implemented by means of a linked list. Because messages can be retrieved in an order different from "first in, first out," the name "message queue" is not appropriate. However, new messages are always put at the end of the linked list.

To send a message, a process invokes the `msgsnd( )` function, passing the following as parameters:

- The IPC identifier of the destination message queue
- The size of the message text
- The address of a User Mode buffer that contains the message type immediately followed by the message text

To retrieve a message, a process invokes the `msgrcv( )` function, passing to it:

- The IPC identifier of the IPC message queue resource
- The pointer to a User Mode buffer to which the message type and message text should be copied
- The size of this buffer
- A value `t` that specifies what message should be retrieved

If the value `t` is 0, the first message in the queue is returned. If `t` is positive, the first message in the queue with its type equal to `t` is returned. Finally, if `t` is negative, the function returns the first message whose message type is the lowest value less than or equal to the absolute value of `t`.

To avoid resource exhaustion, there are some limits on the number of IPC message queue resources allowed (by default, 16), on the size of each message (by default, 8,192 bytes), and on the maximum total size of the messages in a queue (by default, 16,384 bytes). As usual, however, the system administrator can tune these values by writing into the `/proc/sys/kernel/msgmni`, `/proc/sys/kernel/msgmnb`, and `/proc/sys/kernel/msgmax` files, respectively.

The data structures associated with IPC message queues are shown in [Figure 19-2](#). The `msg_ids` variable stores the `ipc_ids` data structure of the IPC message queue resource type; the corresponding `ipc_id_ary` data structure contains an array of pointers to `shmid_kernel` data structures, one item for every IPC message queue resource. Formally, the array stores pointers to `kern_ipc_perm` data structures, but each such structure is simply the first field of the `msg_queue` data structure. All fields of the `msg_queue` data structure are shown in [Table 19-12](#).

Figure 19-2. IPC message queue data structures

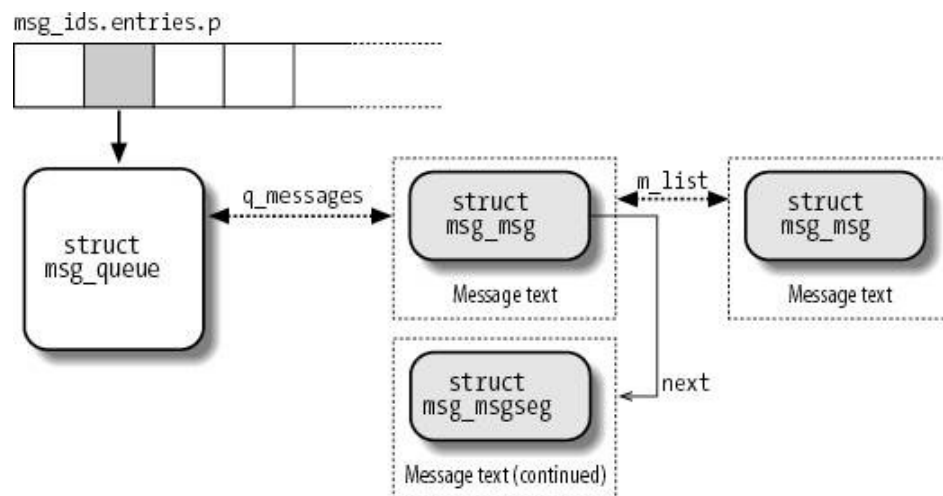


Table 19-12. The msg_queue data structure

Type

Field

Description

struct kern_ipc_perm

q_perm

kern_ipc_perm data structure

long

q_stime

Time of last msgsnd()

long

q_rtime

Time of last msgrcv()

long

q_ctime

Last change time

unsigned long

q_qbytes

Number of bytes in queue

unsigned long

q_qnum

Number of messages in queue

unsigned long

q_qbytes

Maximum number of bytes in queue

int

`q_lspid`

PID of last `msgsnd( )`

`int`

`q_lrpid`

PID of last `msgrcv( )`

`struct list_head`

`q_messages`

List of messages in queue

`struct list_head`

`q_receivers`

List of processes receiving messages

`struct list_head`

`q_senders`

List of processes sending messages

The most important field is `q_messages`, which represents the head (i.e., the first dummy element) of a doubly linked circular list containing all messages currently in the queue.

Each message is broken into one or more pages, which are dynamically allocated. The beginning of the first page stores the message header, which is a data structure of type `msg_msg`; its fields are listed in [Table 19-13](#). The `m_list` field stores the pointers to the previous and next messages in the queue. The message text starts right after the `msg_msg` descriptor; if the message is longer than 4,072 bytes (the page size minus the size of the `msg_msg` descriptor), it continues on another page, whose address is stored in the `next` field of the `msg_msg` descriptor. The second page frame starts with a descriptor of type `msg_msgseg`, which simply includes a `next` pointer storing the address of an optional third page, and so on.

Table 19-13. The `msg_msg` data structure

Type

Field

Description

`struct list_head`

`m_list`

Pointers for message list

long

m_type

Message type

int

m_ts

Message text size

struct msg_msgseg *

next

Next portion of the message

void *

security

Pointer to a security data structure (used by SELinux)

When the message queue is full (either the maximum number of messages or the maximum total size has been reached), processes that try to enqueue new messages may be blocked. The `q_senders` field of the `msg_queue` data structure is the head of a list that includes the pointers to the descriptors of all blocked sending processes.

Even receiving processes may be blocked when the message queue is empty (or the process specified a type of message not present in the queue). The `q_receivers` field of the `msg_queue` data structure is the head of a list of `msg_receiver` data structures, one for every blocked receiving process. Each of these structures essentially includes a pointer to the process descriptor, a pointer to the `msg_msg` structure of the message, and the type of the requested message.

19.3.5. IPC Shared Memory

The most useful IPC mechanism is shared memory, which allows two or more processes to access some common data structures by placing them in an IPC shared memory region. Each process that wants to access the data structures included in an IPC shared memory region must add to its address space a new memory region (see the section "[Memory Regions](#)" in [Chapter 9](#)), which maps the page frames associated with the IPC shared memory region. Such page frames can then be easily handled by the kernel through demand paging (see the section "[Demand Paging](#)" in [Chapter 9](#)).

As with semaphores and message queues, the `shmget ( )` function is invoked to get the IPC identifier of a shared memory region, optionally creating it if it does not already exist.

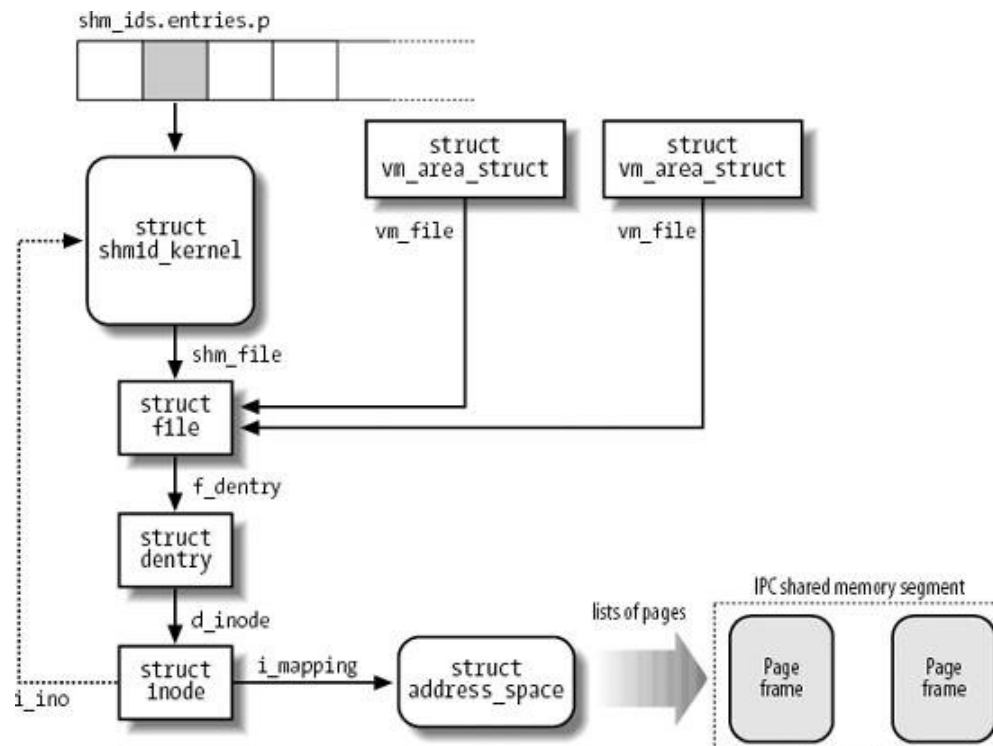
The `shmat ( )` function is invoked to "attach" an IPC shared memory region to a process. It receives as its parameter the identifier of the IPC shared memory resource and tries to add a shared memory region to the

address space of the calling process. The calling process can require a specific starting linear address for the memory region, but the address is usually unimportant, and each process accessing the shared memory region can use a different address in its own address space. The process's Page Tables are left unchanged by `shmat ( )`. We describe later what the kernel does when the process tries to access a page that belongs to the new memory region.

The `shmdt ( )` function is invoked to "detach" an IPC shared memory region specified by its IPC identifier—that is, to remove the corresponding memory region from the process's address space. Recall that an IPC shared memory resource is persistent: even if no process is using it, the corresponding pages cannot be discarded, although they can be swapped out.

As for the other types of IPC resources, in order to avoid overuse of memory by User Mode processes, there are some limits on the allowed number of IPC shared memory regions (by default, 4,096), on the size of each segment (by default, 32 megabytes), and on the maximum total size of all segments (by default, 8 gigabytes). As usual, however, the system administrator can tune these values by writing into the `/proc/sys/kernel/shmmni`, `/proc/sys/kernel/shmmax`, and `/proc/sys/kernel/shmall` files, respectively.

Figure 19-3. IPC shared memory data structures



The data structures associated with IPC shared memory regions are shown in [Figure 19-3](#). The `shm_ids` variable stores the `ipc_ids` data structure of the IPC shared memory resource type; the corresponding `ipc_id_ary` data structure contains an array of pointers to `shm_id_kernel` data structures, one item for every IPC shared memory resource. Formally, the array stores pointers to `kern_ipc_perm` data structures, but each such structure is simply the first field of the `msg_queue` data structure. All fields of the `shm_id_kernel` data structure are shown in [Table 19-14](#).

Table 19-14. The fields in the shmid_kernel data structure

Type

Field

Description

`struct kern_ipc_perm``shm_perm``kern_ipc_perm` data structure`struct file *``shm_file`

Special file of the segment

`int``id`

Slot index of the segment

`unsigned long``shm_nattch`

Number of current attaches

`unsigned long``shm_segsz`

Segment size in bytes

`long``shm_atim`

Last access time

`long``shm_dtim`

Last detach time

`long``shm_ctim`

Last change time

int

shm_cprid

PID of creator

int

shm_lprid

PID of last accessing process

struct user_struct *

mlock_user

Pointer to the `user_struct` descriptor of the user that locked in RAM the shared memory resource (see the section "[The clone\(\), fork\(\), and vfork\(\) System Calls](#)" in [Chapter 3](#))

The most important field is `shm_file`, which stores the address of a file object. This reflects the tight integration of IPC shared memory with the VFS layer in Linux 2.6. In particular, each IPC shared memory region is associated with a file belonging to the `shm` special filesystem (see the section "[Special Filesystems](#)" in [Chapter 12](#)).

Because the `shm` filesystem has no mount point in the system directory tree, no user can open and access its files by means of regular VFS system calls. However, when a process "attaches" a segment, the kernel invokes `do_mmap( )` and creates a new shared memory mapping of the file in the address space of the process. Therefore, files that belong to the `shm` special filesystem have just one file object method, `mmap`, which is implemented by the `shm_mmap( )` function.

As shown in [Figure 19-3](#), a memory region that corresponds to an IPC shared memory region is described by a `vm_area_struct` object (see the section "[Memory Mapping](#)" in [Chapter 16](#)); its `vm_file` field points back to the file object of the file in the special filesystem, which in turn references a dentry object and an inode object. The inode number, stored in the `i_ino` field of the inode, is actually the slot index of the IPC shared memory region, so the inode object indirectly references the `shmid_kernel` descriptor.

As usual for every shared memory mapping, page frames are included in the page cache through an `address_space` object, which is embedded in the inode and referenced by the `i_mapping` field of the inode (you might also refer to [Figure 16-2](#)); in case of page frames belonging to an IPC shared memory region, the methods of the `address_space` object are stored in the `shmem_aops` global variable.

19.3.5.1. Swapping out pages of IPC shared memory regions

The kernel has to be careful when swapping out pages included in shared memory regions, and the role of the swap cache is crucial (this topic was already discussed in the section "[The Swap Cache](#)" in [Chapter 17](#)).

Pages of an IPC shared memory region are swappable and not syncable (see [Table 17-1](#) in [Chapter 17](#)) because they map a special inode that has no image on disk. Thus, in order to reclaim a page of an IPC shared memory region, the kernel must write it into a swap area. Because an IPC shared memory region is persistent that is, its pages must be preserved even when the segment is not attached to any process the kernel cannot simply

discard these pages even when they are no longer used by any process.

Let us see how the PFRA performs the reclaiming of a page frame used by an IPC shared memory region. Everything is done as described in the section "[Low On Memory Reclaiming](#)" in [Chapter 17](#), until the page is considered by `shrink_list()`. Because this function does not include any special check for pages of IPC shared memory regions, it ends up invoking the `try_to_unmap()` function to remove every reference to the page frame from the User Mode address spaces; as explained in the section "[Reverse Mapping](#)" in [Chapter 17](#), the corresponding page table entries are simply cleared.

Next, the `shrink_list()` function checks the `PG_dirty` flag of the page and invokes `pageout()` page frames of IPC shared memory regions are marked dirty when they are allocated, thus `pageout()` is always invoked. In turn, the `pageout()` function invokes the `writepage` method of the `address_space` object of the mapped file.

The `shmem_writepage()` function, which implements the `writepage` method for IPC shared memory regions' pages, essentially allocates a new page slot in a swap area, and moves the page from the page cache to the swap cache (it's just a matter of changing the owner `address_space` object of the page). The function also stores the swapped-out page identifier in a `shmem_inode_info` structure that embodies the IPC memory region's inode object, and it sets again the `PG_dirty` flag of the page. As shown in [Figure 17-5](#) in [Chapter 17](#), the `shrink_list()` function checks the `PG_dirty` flag and breaks the reclaiming procedure by leaving the page in the inactive list.

Sooner or later, the page frame will be processed again by the PFRA. Once again, the `shrink_list()` function will try to flush the page to disk by invoking `pageout()`. This time, however, the page is included in the swap cache, thus it is "owned" by the `address_space` object of the swapping subsystem, `swapper_space`. The corresponding `writepage` method, `swap_writepage()`, effectively starts the write operation into the swap area (see the section "[Swapping Out Pages](#)" in [Chapter 17](#)). Once `pageout()` terminates, `shrink_list()` verifies that the page is now clean, removes it from the swap cache, and releases it to the buddy system.

19.3.5.2. Demand paging for IPC shared memory regions

The pages added to a process by `shmat()` are dummy pages; the function adds a new memory region into a process's address space, but it doesn't modify the process's Page Tables. Moreover, as we have seen, pages of an IPC shared memory region can be swapped out. Therefore, these pages are handled through the demand paging mechanism.

As we know, a Page Fault occurs when a process tries to access a location of an IPC shared memory region whose underlying page frame has not been assigned. The corresponding exception handler determines that the faulty address is inside the process address space and that the corresponding Page Table entry is null; therefore, it invokes the `do_no_page()` function (see the section "[Demand Paging](#)" in [Chapter 9](#)). In turn, this function checks whether the `nopage` method for the memory region is defined. That method is invoked, and the Page Table entry is set to the address returned from it (see also the section "[Demand Paging for Memory Mapping](#)" in [Chapter 16](#)).

Memory regions used for IPC shared memory always define the `nopage` method. It is implemented by the `shmem_nopage()` function, which performs the following operations:

1. Walks the chain of pointers in the VFS objects and derives the address of the inode object of the IPC shared memory resource (see [Figure 19-3](#)).
2. Computes the logical page number inside the segment from the `vm_start` field of the memory region descriptor and the requested address.

3. Checks whether the page is already included in the page cache; if so, terminates by returning the address of its descriptor.
4. Checks whether the page is included in the swap cache and is up-to-date; if so, terminates by returning the address of its descriptor.
5. Checks whether the `shmem_inode_info` that embodies the inode object stores a swapped-out page identifier for the logical page number. If so, it performs a swap-in operation by invoking `read_swap_cache_async( )` (see the section "[Swapping in Pages](#)" in [Chapter 17](#)), waits until the data transfer completes, and terminates by returning the address of the page descriptor.
6. Otherwise, the page is not stored in a swap area; therefore, the function allocates a new page from the buddy system, inserts it into the page cache, and returns its address.

The `do_no_page( )` function sets the entry that corresponds to the faulty address in the process's Page Table so that it points to the page frame returned by the method.

19.4. POSIX Message Queues

The POSIX standard (IEEE Std 1003.1-2001) defines an IPC mechanism based on message queues, which is usually known as POSIX message queues . They are much like the System V IPC's message queues already examined in the section "[IPC Messages](#)" earlier in this chapter. However, POSIX message queues sport a number of advantages over the older queues:

- A much simpler file-based interface to the applications
- Native support for message priorities (the priority ultimately determines the position of the message in the queue)
- Native support for asynchronous notification of message arrivals, either by means of signals or thread creation
- Timeouts for blocking send and receive operations

POSIX message queues are handled by means of a set of library functions, which are shown in [Table 19-15](#).

Table 19-15. Library functions for POSIX message queues

Function names	Description
<code>mq_open()</code>	Open (optionally creating) a POSIX message queue
<code>mq_close()</code>	Close a POSIX message queue (without destroying it)
<code>mq_unlink()</code>	Destroy a POSIX message queue
<code>mq_send() ,</code>	Send a message to a POSIX message queue; the latter function defines a time limit for the operation
<code>mq_timedsend()</code>	Fetch a message from a POSIX message queue; the latter function defines a time limit for the operation
<code>mq_receive() ,</code>	Establish an asynchronous notification mechanism for the arrival of messages in an empty POSIX message queue
<code>mq_timedreceive()</code>	Respectively get and set attributes of a POSIX message queue (essentially, whether the send and receive operations should be blocking or nonblocking)
<code>mq_notify()</code>	
<code>mq_getattr() ,</code>	
<code>mq_setattr()</code>	

Let's see how an application typically makes use of these functions. As a first step, the application invokes the `mq_open( )` library function to open a POSIX message queue. The first argument of the function is a string specifying the name of the queue; it is similar to a filename, and indeed it must start with a slash (/). The library function accepts a subset of the flags of the `open( )` system call: `O_RDONLY`, `O_WRONLY`,

`O_RDWR`, `O_CREAT`, `O_EXCL`, and `O_NONBLOCK` (for nonblocking send and receive operations). Notice that the application may create a new POSIX message queue by specifying the `O_CREAT` flag. The `mq_open( )` function returns a descriptor for the queue much like the file descriptor returned by the `open( )` system call.

Once a POSIX message queue has been opened, the application may send and receive messages by using the library functions `mq_send( )` and `mq_receive( )`, passing to them the queue descriptor returned by `mq_open( )`. The application may also make use of `mq_timedsend( )` and `mq_timedreceive( )` to specify the maximum time that the application will spend waiting for the send or receive operation to complete.

Rather than blocking in `mq_receive( )` or continuously polling the message queue if the `O_NONBLOCK` flag was specified, the application might also establish an asynchronous notification mechanism by executing the `mq_notify( )` library function. Essentially, the application may require that when a message is inserted in an empty queue, either a signal is sent to a selected process, or a new thread is created.

Finally, when the application has finished using the message queue, it invokes the `mq_close( )` library function; passing to it the queue descriptor. Notice that this function does not destroy the queue, exactly as the `close( )` system call does not remove a file. To destroy a queue, the application makes use of the `mq_unlink( )` function.

The implementation of POSIX message queues in Linux 2.6 is simple and straightforward. A special filesystem named `mqueue` (see the section "[Special Filesystems](#)" in [Chapter 12](#)) has been introduced, which contains an inode for each existing queue. The kernel offers a few system calls, which roughly correspond to the library functions listed in [Table 19-15](#) earlier: `mq_open( )`, `mq_unlink( )`, `mq_timedsend( )`, `mq_timedreceive( )`, `mq_notify( )`, and `mq_getsetattr( )`. These system calls act transparently on the files of the `mqueue` filesystem, thus much of the job is done by the VFS layer. For example, notice that the kernel does not offer a `mq_close( )` function: in fact, the queue descriptor returned to the application is effectively a file descriptor, therefore the `mq_close( )` library function can simply execute the `close( )` system call to do its job.

The `mqueue` special filesystem must not necessarily be mounted over the system directory tree. However, if it is mounted, a user can create a POSIX message queue by touching a file in the root directory of the filesystem; she can also get information about the queue by reading the corresponding file. Finally, an application can use `select( )` and `poll( )` to be notified about changes in the queue state.

Each queue is described by an `mqueue_inode_info` descriptor, which embodies the inode object associated with the file in the `mqueue` special filesystem. When a POSIX message queue system call receives a queue descriptor as parameter, it invokes the VFS's `fget( )` function to derive the address of the corresponding file object; next, the system call gets the inode object of the file in the `mqueue` filesystem, and finally the address of the `mqueue_inode_info` descriptor that contains the inode object.

The pending messages in a queue are collected in a singly linked list rooted at the `mqueue_inode_info` descriptor; each message is represented by a descriptor of type `msg_msg` exactly the same descriptor used for the System V IPC's messages described in the section "[IPC Messages](#)" earlier in this chapter.